

How render timing actually works

This hook isn't really about hooks. It's about the gap between when a render happens and when an effect fires.

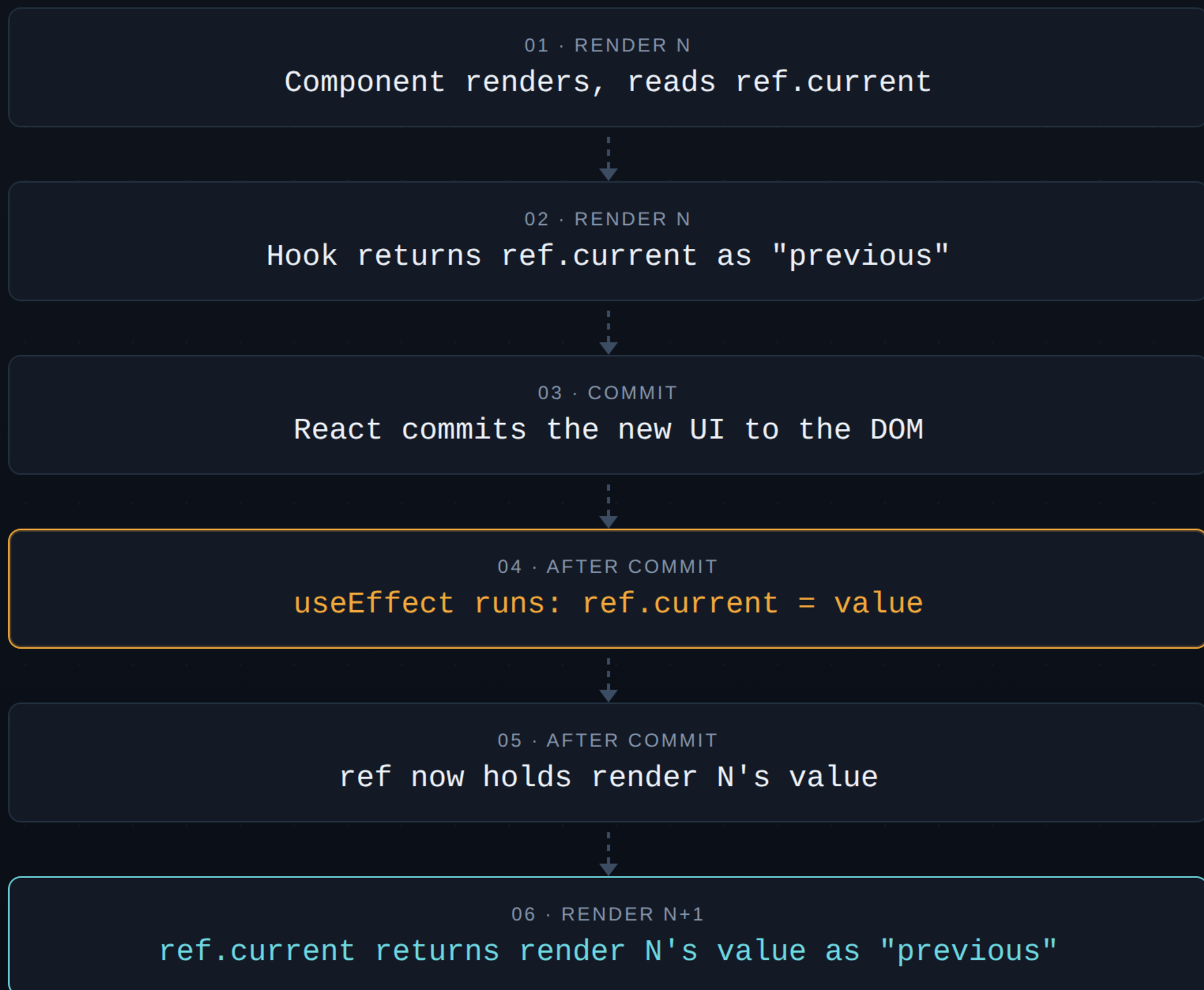
- 1 React renders your component top to bottom, computing what the UI should look like for this pass.
- 2 A **ref** set during the last render still holds its old value right now. Reading `.current` doesn't wait for anything.
- 3 Only after React commits the new UI to the DOM do your **useEffect** callbacks run.
- 4 That's the whole trick. Read before the effect runs and you get old data. Read after, you get new.

Reaching for `useState` instead of `useRef` breaks this. Updating state schedules another render, and now you're chasing the value you just tried to capture.

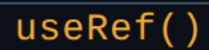
The algorithm, boiled down

Six steps, from the render that reads the ref to the effect that rewrites it.

`<usePrevious>`



A container that survives renders

useRef() usePrevious.js

```
function usePrevious(value) {  
  const ref = useRef();  
}
```

- > useRef gives you a mutable container that persists across renders without causing new ones.
- > Unlike useState, writing to **ref.current** never triggers a re-render.
- > No initial value is passed, so on the very first render it's undefined. That's fine, there's no "previous" yet.

Read it before anything updates it

```
return ref.current
```

```
● ● ● usePrevious.js
```

```
function usePrevious(value) {  
  const ref = useRef();  
  return ref.current;  
}
```

- > This return runs during render, synchronously, before any effect has a chance to fire.
- > Effects are deferred until after commit, so **ref.current** here still holds last render's value.
- > That gap between render and commit is the entire reason this hook works.

Write it back only after commit

`useEffect()``● ● ● usePrevious.js`

```
useEffect(() => {  
  ref.current = value;  
}, [value]);
```

- > This callback doesn't run during render. React defers it until after the DOM commits.
- > It overwrites **ref.current** with this render's value, but only after that value was already returned as "previous."
- > The dependency array means it only re-runs when value actually changes, not on every render.

Six lines, one ordering trick

`usePrevious()``● ● ● usePrevious.js`

```
function usePrevious(value) {  
  const ref = useRef();  
  useEffect(() => {  
    ref.current = value;  
  }, [value]);  
  return ref.current;  
}
```

- > One ref, one effect, one return. Nothing else is doing any work here.
- > Reading happens before the effect runs, writing happens after. That ordering is the whole trick.
- > Say that ordering out loud in an interview, and you've answered the question they're actually asking.